

Manual del Proyecto: ESP32 Kids Lab

Documentación de Proyecto

12 de julio de 2026

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
3. Componentes de Hardware y Mapa de Pines	2
3.1. Actuadores	2
3.2. Sensores	3
4. Descripción del Firmware	3
4.1. API REST (Endpoints)	3
5. Interfaz de Usuario (Frontend)	3
6. Conclusión	4

1. Introducción

El proyecto **ESP32 Kids Lab** es un laboratorio educativo basado en el microcontrolador ESP32. Su objetivo principal es enseñar conceptos básicos de electrónica y programación mediante la interacción interactiva con sensores y actuadores a través de una interfaz web.

El sistema configura el ESP32 en modo Access Point (Punto de Acceso) y aloja un servidor web que permite a los usuarios interactuar con el hardware en tiempo real sin necesidad de una infraestructura de red externa.

2. Arquitectura del Sistema

El proyecto se divide en dos componentes principales:

- **Firmware (Backend):** Escrito en C++ utilizando el framework Arduino y PlatformIO. Se encarga de leer los sensores, controlar los actuadores y servir la página web.
- **Interfaz Web (Frontend):** Archivos HTML, CSS y JavaScript alojados en el sistema de archivos LittleFS del ESP32. Estos archivos proveen la interfaz gráfica para el usuario.

3. Componentes de Hardware y Mapa de Pines

El ESP32 interactúa con múltiples componentes electrónicos. A continuación se detalla la asignación de pines (GPIO):

3.1. Actuadores

- **LED Rojo:** GPIO 1
- **LED Verde:** GPIO 23
- **LED Azul:** GPIO 19
- **LED Amarillo:** GPIO 3
- **Relé (Relay):** GPIO 0
- **Bocina (Buzzer):** GPIO 12
- **Matriz LED WS2812 (NeoPixel):** GPIO 13 (16 píxeles)
- **Display de 7 Segmentos (TM1637):** CLK → GPIO 27, DIO → GPIO 14

3.2. Sensores

- **Sensor Ultrasónico (Distancia):** Trig → GPIO 15, Echo → GPIO 2
- **Sensor de Luz (LDR):** GPIO 35
- **Sensor de Temperatura (DS18B20 - 1-Wire):** GPIO 4
- **Potenciómetro:** GPIO 36
- **Sensor de Inclinación (Tilt):** GPIO 39

4. Descripción del Firmware

El código principal se encuentra en `src/main.cpp`. Utiliza las siguientes librerías principales:

- `ESPAsyncWebServer` y `AsyncTCP`: Para el servidor web asíncrono.
- `LittleFS`: Para montar el sistema de archivos y servir la interfaz web.
- `ArduinoJson`: Para la serialización y deserialización de datos JSON entre el frontend y el backend.
- `Adafruit NeoPixel`: Para el control de la matriz LED.
- `TM1637Display`: Para controlar el display de 7 segmentos.
- `DallasTemperature` y `OneWire`: Para la lectura del sensor DS18B20.

4.1. API REST (Endpoints)

La comunicación entre la interfaz web y el ESP32 se realiza a través de peticiones HTTP.

- `GET /api/sensors`: Retorna un objeto JSON con los valores actuales de los sensores (distancia, luz, temperatura, inclinación, potenciómetro).
- `POST /api/actuators`: Recibe un objeto JSON para encender o apagar los LEDs básicos, el relé o el buzzer.
- `POST /api/display`: Recibe un número para mostrar en el display de 7 segmentos o un comando para limpiarlo.
- `POST /api/matrix`: Recibe datos de colores (RGB) para iluminar píxeles específicos o todos los píxeles de la matriz NeoPixel.

5. Interfaz de Usuario (Frontend)

La interfaz gráfica está diseñada para ser amigable y accesible (archivos en la carpeta `data/`). A través de la comunicación asíncrona mediante JavaScript (`app.js`), la página realiza peticiones periódicas para actualizar el estado de los sensores y enviar los comandos a los actuadores en tiempo real cuando el usuario interactúa con los controles.

6. Conclusión

El ESP32 Kids Lab es una plataforma versátil para la educación STEM. Demuestra cómo integrar de manera eficiente múltiples sensores analógicos y digitales, así como protocolos de comunicación como 1-Wire y señales PWM o específicas como WS2812, todo en un entorno unificado con conectividad WiFi.